

《计算机网络课程设计》任务书

1. 实验环境和分组要求

- 操作系统：Windows 或 Linux 系统。
- 编程语言：C
- 分组：1-3 人。多人合作的，提交的程序必须是小组所有同学都能消化的部分，能经得起质疑。

2. 基本任务

设计并实现一个 DNS 中继服务器程序，读入“域名-IP 地址”对照表，当客户端查询域名对应的 IP 地址时，用域名检索该对照表，可能有三种检索结果：

- 检索结果为 ip 地址 0.0.0.0，则向客户端返回“域名不存在”的报错消息，而不是返回 IP 地址为 0.0.0.0，实现不良网站拦截功能。
- 检索结果为普通 IP 地址，则向客户返回这个地址，实现 DNS 服务器功能。
- 表中未检到该域名，则向因特网 DNS 服务器发出查询，并将结果返给客户端，实现 DNS 中继功能。

要求根据 DNS 协议文本，按照协议要求，实现与 Windows 或其他系统的互联互通。

注意：

1. 多客户端并发：允许多个客户端（可能会位于不同的多个计算机）的并发查询，即：允许第一个查询尚未得到答案前就启动处理另外一个客户端查询请求（DNS 协议头中 ID 字段的作用），需要进行消息 ID 的转换

2. 超时处理：由于 UDP 的不可靠性，考虑求助外部 DNS 服务器（中继）却不能得到应答或者收到迟到应答的情形

3. 其他更高级的功能

例如但不限于下列功能：

- 实现 AAAA、MX、CNAME、PTR 等类型的查询
- 用数据库存储 DNS 资源记录
- 并发式查询机制
- 查询算法的优化
- 实现 LRU 机制的 Cache 缓存
- Windows/Linux 源程序的一致性能

4. Windows 系统 DNS 中继服务器运行

运行步骤如下：

- (1) 使用 `ipconfig/all`，记下当前 DNS 服务器，例如 202.106.0.20
- (2) 使用本机的网络配置界面，将 DNS 设置为 127.0.0.1(本地主机)
- (3) 运行你的 `dnsrelay` 程序(在你的程序中把外部 `dns` 服务器设为前面记下的 202.106.0.20)
- (4) 正常使用 `ping`、`ftp`、浏览器等功能，名字解析工作正常
- (5) 局域网上的其他计算机（Windows 或 Linux）将域名服务器指向 DNS 中继服务器的 IP 地址，`ping` 和浏览器等均能正常工作

可用的其它 DOS 命令：

- ✧ `nslookup www.bupt.edu.cn`：向名字服务器询问名字 `www.bupt.edu.cn` 的地址
- ✧ `nslookup`：每输入一个名字，给出解析结果
- ✧ `ipconfig/displaydns`：察看当前 `dns cache` 的内容以确认程序执行结果的正确性
- ✧ `ipconfig/flushdns`：清除 `dns cache` 中缓存的所有 DNS 记录

5. 参考实现

命令语法

`dnsrelay [-d | -dd] [dns-server-ipaddr] [filename]`

`dnsrelay`

无调试信息输出

使用默认名字服务器 202.106.0.20

使用默认配置文件(当前目录下 `dnsrelay.txt`)

`dnsrelay -d 192.168.0.1 c:\dns-table.txt`

调试信息级别 1（仅输出时间坐标，序号，客户端 IP 地址，查询的域名）

使用指定的名字服务器 192.168.0.1

使用指定的配置文件 `c:\dns-table.txt`

`dnsrelay -dd 202.99.96.68`

调试信息级别 2（输出冗长的调试信息）

使用指定的名字服务器 202.99.96.68

使用默认配置文件(当前目录下 `dnsrelay.txt`)

6. 撰写课程设计报告

报告应包含下列基本内容：

- 《课程设计报告封面》，填写基本信息及课程设计内容
- 系统的功能设计
- 模块划分和软件流程图
- 测试用例以及运行结果
- 调试中遇到并解决的问题
- 总结和心得体会

7. 以往课程设计程序中存在的问题

下列问题供参考：

- (1) 格式问题，不够整洁，该有的空格必须要求要有，不该有的空格，必须没有。注意缩进以及必要的换行，太多的缩进，是代码整体设计布局有问题。所完成的代

码耗费了你不少心血，珍惜自己的劳动，要爱惜代码中每个字符。版面必须要整洁。

- (2) 程序需要使用几个 `socket`? 需要自己调研和尝试，并给出合理的理由。
- (3) 忙等待: CPU 每秒钟做成千上百万次几乎不会命中但偶尔会命中的探测，1 个 CPU 核忙死在你的程序里，实际也许只需要这个 CPU 的 0.001% 的算力，你却用了 100%。应检查 CPU 占用率，考虑一下，1 秒钟对付 10 个查询 OK，能对付 10 万个吗?
- (4) 用定时解决等待服务器应答的问题: 定时器长了或短了都有问题，把这些问题透明传递给客户端是一种可选的方法。
- (5) 基本不划分子程序，一个 `main` 函数流水到底，只有极少数子程序。类似 ID 转换之类算法所需要的数据结构裸露在外，缺少合理的封装。整个工程只有一个 .C 文件，没有降低模块之间耦合度的意识。
- (6) 可读性问题: 无聊的注释太多完全不妨碍代码可读性依然很差。注释中甚至有 BUG，骗人的注释还不如没有。变量和函数取名太随意，影响可读性，名字跟实际做得甚至不一样，误导人，可读性为负值? 取个合理的名字需要花时间花精力思考。
- (7) Windows/Linux 兼容考虑: 最差方案，维护两份源代码。第二差方案，只有一份源代码文件，有散落在核心代码里的多处条件编译。应该把与平台相关的部分集中在一起，核心代码中使用你自己抽象出的平台功能，比如核心代码调用 `socket_init()`，在 Linux 把它定义为宏，为空；在 Windows，做个函数，完成平台要求的初始化。
- (8) 调试信息的输出缺少封装，与调试信息有关的过多的语法符号和代码行，喧宾夺主，影响主体功能的展示。不需要调试的时候，这些调试信息应几乎不占用 CPU，封装成宏和相关子程序较为合理。
- (9) Cache 中缓存的表应该是动态的，从文件获取的静态表，查找或者其他操作，算法 $O(n)$ 效率，或者选择的算法不当，这样的效率不可接受。
- (10) 为什么不把 `cache` 和静态表的查找合并为一个程序做? 缺乏对问题的归纳和抽象。

注意：复制后稍改动存在两份代码与只有一份代码适应两种情况，区别太大了，要极力避免复制一片代码再小修小补，程序中存在两处类似的代码。

- (11)代码修改了协议的基本特征，失去了透明性。服务器超时不应答向服务器重传 query 不符合协议，没有你的中继，客户端得到的服务不是这样的。
- (12)死锁问题：等不到服务器的应答，死等下去，程序不再从客户端收数据。
- (13)多线程缺乏正确的同步措施，如：一个线程收下数据排队，另一个线程读取队列处理。读取队列的线程处于忙等待状态占掉一个 CPU 核 100%的算力。
- (14)链表问题：建议摒弃教科书中的方法，链表有三种成熟的模式(两种双向链表 list_head 和 hlist，以指针的指针方式操作单链表)，网上很容易找到的，只有教科书上找不到。深刻分析这三种模式，比较一下为什么你使用的很朴素很容易理解的链表操作模式是创生 BUG 的利器，看起来那么蹩脚的模式却得到推崇。提示：与第 10 条有关。
- (15)没有正确采用面向对象的程序设计思想。id 转换，cache 表，等等，要把它想象成独立的对象（事实就是这样的），使用者和设计者要有界面感，类似七层协议模型的 interface、service、implementation，对使用者暴露的东西越少越好，减少模块之间的耦合度。设想一下你的程序，把 cache 方案彻底推翻，采用完全不同的数据结构，更换一个更复杂更高效的算法，甚至会引入硬件加速，你的主程序不需要做任何改动，或者改动很少。改动越少说明你做得越好。经典案例：TCP 协议一直在改进，使用 TCP 协议的应用程序除了觉得速度更快了之外完全无感，因为 interface 和 service 没变，变的是 implementation。C 语言的文件和 socket 访问也是面向对象式的，围绕对象有一组方法，内部状态是隐藏的。心中有对象，用 C 可以写出面向对象的程序，这样的话 Linux 内核源代码就是；心中无对象，用 C++和 Java 也一样写出比 C 还 C 的程序。

8. 参考资料

TCP/IP 网络互连技术 卷 3 , Douglas E. Comer,清华大学出版社, 1999.10

RFC1035 DOMAIN NAMES-IMPLEMENTATION AND SPECIFICATION

9. 《课程设计报告》首页

北京邮电大学课程设计报告

课程设计名称		学 院		指导教师	
班 级	班内序号	学 号		学生姓名	成绩
课 程 设 计 内 容	简要介绍课程设计的主要内容，包括课程设计教学目的、基本内容、实验方法和团队分工等				
学生 课程设计 报告 (附页)					
课 程 设 计 成 绩 评 定	评语：				
	成绩： 指导教师签名： 年 月 日				

注：评语要体现每个学生的工作情况，可以加页。